The AI Telco Engineer: Toward Autonomous Discovery of Wireless Communications Algorithms

Fayçal Aït Aoudia, Jakob Hoydis, Sebastian Cammerer,
Lorenzo Maggi, Gian Marti, and Alexander Keller

***Abstract*—Agentic AI is rapidly transforming the way research is conducted, from prototyping ideas to reproducing results found in the literature. In this paper, we explore the ability of agentic AI to autonomously design wireless communication algorithms. To that aim, we implement a dedicated framework that leverages large language models (LLMs) to iteratively generate, evaluate, and refine candidate algorithms. We evaluate the framework on three tasks spanning the physical (PHY) and medium access control (MAC) layers: statistics-agnostic channel estimation, channel estimation with known covariance, and link adaptation. Our results show that, in a matter of hours, the framework produces algorithms that are competitive with and, in some cases, outperforming conventional baselines. Moreover, unlike neural network-based approaches, the generated algorithms are fully explainable and extensible. This work represents a first step toward the autonomous discovery of novel wireless communication algorithms, and we look forward to the progress our community makes in this direction.**

I. Introduction

The pace at which agentic AI capabilities are advancing has begun to fundamentally alter research workflows. With new agentic frameworks being released on a daily basis, many researchers have already integrated such tools into their workflow. In particular, coding agents such as Claude Code, GPT Codex, and Cursor make it easier than ever to rapidly prototype ideas, removing the burden of complex software toolchains that was traditionally associated with the development and evaluation of new algorithmic concepts.

Reproducing results from the literature has also become significantly more accessible thanks to dedicated agentic frameworks, such as DeepCode [1], which aim to precisely reproduce the results of a publication solely from the document itself. With such tools, there is little reason not to implement baselines when benchmarking new methods. One should also highlight deep search tools, which autonomously build research reports by exploring the literature based on a user query. Examples of such tools include GPT Researcher [2], as well as the deep search capabilities offered by OpenAI, Anthropic Claude, Google Gemini, Perplexity, and others.

In this context, it is only natural to consider how agentic AI could impact research in wireless communications. We tackle this vast question by exploring the ability of agentic AI to autonomously design wireless communication algorithms. To that aim, we have built a dedicated agentic AI framework, *The AI Telco Engineer*, that leverages Sionna [3] to evaluate algorithms in link- and system-level simulations. The framework includes a tool for the agent to access the Sionna API documentation, developer guides, and tutorials. Code is executed in containerized environments, referred to as *workspaces*, in which filesystem editing tools are also available to the agent. Using this framework, we show that agentic AI can design wireless communication algorithms that are competitive with, and in some cases outperform, conventional approaches.

*All authors are with NVIDIA. Contact: `faitaoudia@nvidia.com`.
Repository: https://github.com/nvlabs/the-ai-telco-engineer.

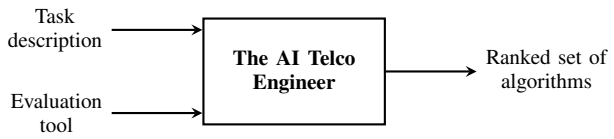


Fig. 1. The framework from an input/output perspective. The user provides a task, i.e., a problem description paired with an evaluation tool, and the framework returns a ranked set of algorithms.

Similar frameworks for program evolution exist, such as ShinkaEvolve [4] and OpenEvolve [5]. However, we have opted to build a custom framework in order to freely experiment with agentic AI strategies best suited to wireless communication algorithm design. We plan to continue improving our framework to further specialize it for this domain and enhance its performance.

The remainder of this paper is organized as follows. Section II presents the architecture of the proposed framework. Section III describes the considered tasks and discusses the obtained results. Section IV concludes the paper.

II. Framework Architecture

An input/output view of the framework is shown in Fig. 1. The framework takes as input a *task*, which consists of a description of the problem for which algorithms are to be generated as well as an *evaluation tool* used by the agent to assess a generated algorithm against a user-defined metric. The evaluation tool is immutable to the agents, which can only invoke it, in order to prevent metric gaming, i.e., the agent modifying the evaluation procedure itself to artificially inflate its score rather than genuinely improving the algorithm. As output, the framework produces not a single algorithm but a set of algorithms ranked according to the user-defined metric.

Fundamentally, the framework builds on the idea of using LLMs to explore the space of algorithms solving a well-defined problem [6]–[8], by iteratively expanding a population

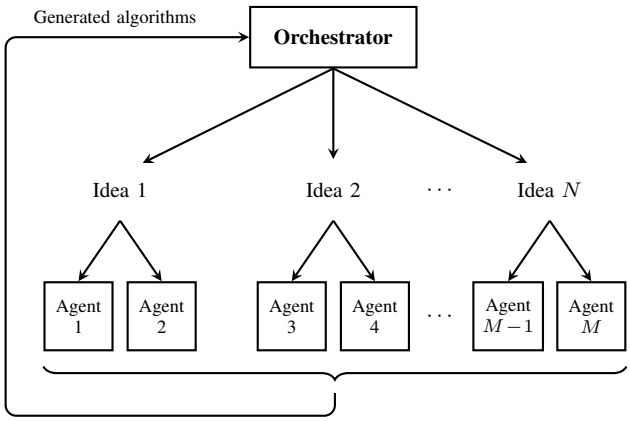


Fig. 2. Overview of the idea-driven iterative optimization loop. The orchestrator generates ideas that are distributed to parallel agents, each operating in its own workspace. Algorithms flow back to the orchestrator to inform the next iteration. In this figure, each idea is assigned to two agents.

of candidate solutions. The remainder of this section details the architecture of the optimization loop, the agent workflow, and the tools available to agents.

A. Overview

The framework follows a two-tier architecture: an *orchestrator* LLM drives the optimization process, while a population of *agent* LLMs implements and refines solutions in parallel. An overview of the architecture is shown in Fig. 2.

The optimization proceeds over multiple iterations. At each iteration, the orchestrator proposes N distinct algorithmic ideas, and a population of M agents is distributed across these ideas, with M/N agents assigned to each. Agents work independently and in parallel, each within its own isolated containerized environment referred to as a *workspace*. Each idea is typically assigned to multiple agents to exploit LLM stochasticity: different agents assigned to the same idea will likely produce different implementation variants (as long as the LLM's temperature is set sufficiently high), effectively exploring multiple realizations of a single approach. This diversity reduces the risk of a promising idea being discarded due to a single poor implementation.

After an iteration completes, the orchestrator reviews the outcomes and proposes a new set of ideas for the next iteration. The orchestrator considers solutions from all previous iterations, not only the most recent one. To avoid saturating the context window, solutions produced by agents are first summarized in natural language to highlight their key aspects, and these summaries are provided to the orchestrator LLM alongside the corresponding metrics. Ideas proposed for subsequent iterations can be entirely novel approaches, refinements of previously explored ideas, or combinations of multiple prior ideas; the orchestrator decides how to proceed based on the accumulated results. The full optimization loop is summarized in Algorithm 1.

Algorithm 1 Idea-driven iterative optimization

Require: Task description $\mathcal{T}$, evaluation tool $\mathcal{E}$, number of ideas N, population size M, number of iterations G

$\mathcal{S} \leftarrow \emptyset$
for $g = 1$ **to** G **do**
  ▷ *$\mathcal{I}_g$: set of N ideas for iteration g*
  if $g = 1$ **then**
    $\mathcal{I}_g \leftarrow \text{Orchestrator}(\mathcal{T}, N)$
  else
    $\mathcal{I}_g \leftarrow \text{Orchestrator}(\mathcal{T}, N, \text{summaries}(\mathcal{S}))$
  end if
  for all idea $I_k \in \mathcal{I}_g$ **do**
    for $j = 1$ **to** M/N **do**
      Spawn agent with $(\mathcal{T}, I_k, \mathcal{E})$
    end for
  end for
  ▷ *Run all agents in parallel*
  for all completed agent with solution s **do**
    $\text{metric} \leftarrow \mathcal{E}(s)$
    $\text{summary} \leftarrow \text{Orchestrator.summarize}(s, I_k)$
    $\mathcal{S} \leftarrow \mathcal{S} \cup \{(s, \text{metric}, \text{summary})\}$
  end for
end for
return $\mathcal{S}$ ranked by metric

B. Agent Workflow

Each agent operates within an isolated Docker container and is given the task description together with an assigned algorithmic approach and, optionally, reference code from prior iterations. As illustrated in Fig. 3, each agent is an LLM with access to a set of tools, all running inside a containerized workspace.

The agent follows a structured two-file workflow. A file `draft.py` serves as a scratch pad for experimentation: the agent writes, edits, and evaluates code in this file. A second file, `solution.py`, acts as a vault for the best result found so far. The agent is instructed to copy `draft.py` to `solution.py` only when a new best metric is achieved, ensuring that progress is preserved even if the agent is interrupted. The agent iteratively implements its approach, invokes the evaluation tool to measure performance, and refines the solution until it runs out of ideas or time. A configurable timeout bounds each agent's execution. At the end of each run, whether by normal completion or timeout, a post-run evaluation is automatically performed on the final `solution.py`, and the result is recorded.

Within its workspace, an agent can execute arbitrary code, carry out file management operations (read, write, edit, copy, and delete files), and install packages. A workspace only lives for the duration of a single iteration. At every iteration, a fresh workspace is spawned for each agent, ensuring that corruption of a workspace does not impact subsequent iterations.

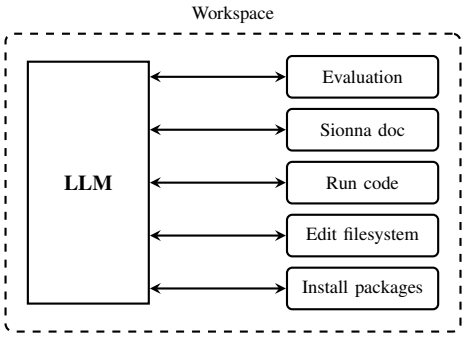


Fig. 3. Agent runtime environment. Each agent is an LLM running inside an isolated containerized workspace with access to a set of tools.

C. Sionna Documentation Tool

The framework uses Sionna [3] to perform link-level and system-level simulations. To enable agents to write correct Sionna code, we provide them with tools to access the Sionna API documentation, tutorials, and developer guides through three operations. *Search* performs semantic search over the documentation using a FAISS [9] vector store with a cross-encoder reranker; the vector store is built once from Sionna API docstrings and online tutorials, then memory-mapped across worker processes. *Help* retrieves the full docstring and signature of any Sionna symbol by executing introspection code within the workspace. *List* enumerates available classes and functions within a given Sionna module. These tools allow agents to use Sionna blocks and utilities, and to understand Sionna data structures such as `ResourceGrid` and `StreamManagement` when required.

D. Evaluation Tool

A task-specific evaluation tool is provided as a pluggable component by the user. It executes the agent's code and returns a structured output with three components. First, a binary success/failure flag indicates whether the evaluation completed successfully. Because agents generate code autonomously, execution failures such as runtime errors or incompatible outputs are common, particularly in early iterations. A failure can also be triggered by the evaluation tool itself when a constraint is not satisfied, as discussed below. In case of failure, the error stack trace or a diagnostic message is returned to help the agent debug or refine its code. Second, a scalar primary metric serves as the optimization objective. Third, the evaluation tool may return optional auxiliary information, such as additional metrics or diagnostic data, which is provided to the agent as feedback but is not used for ranking. This design accommodates problems with multiple objectives: the evaluation tool can encode constraints into the success flag, for instance, marking a run as failed if a secondary metric violates a prescribed threshold, so that the primary metric is optimized only over the feasible set. As noted above, the evaluation tool is not editable by the agents, preventing metric gaming. Moreover, each call to the evaluation tool is subject to a timeout, which prevents any single evaluation from blocking the optimization loop and also serves as an implicit constraint on the computational complexity of the generated algorithms.

III. Experimental Evaluation

To evaluate the framework, we consider three tasks: two PHY layer tasks involving channel estimation, and one MAC layer task involving link adaptation. We emphasize that the framework is not specific to these tasks, as it could be applied to any wireless communication problem for which an evaluation tool can be defined.

For all tasks, two off-the-shelf LLMs are considered: a closed-weights model, GPT 5.4, and an open-weights model, GPT-OSS 120B. The framework was run once per model for each task, resulting in two independent sets of generated algorithms per task. Both the orchestrator and the agents were configured to use the same model within a given run. The framework was configured to execute $G = 10$ iterations, each with $M = 20$ agents running in parallel and the orchestrator generating $N = 5$ ideas per iteration. Each idea was therefore assigned to 4 agents. The timeout per agent was set to 20 minutes, and agents had access to GPUs for executing Python code using Sionna. Due to the inherent stochasticity of LLMs, different runs of the same task produce different sets of algorithms with varying performance; however, the best-performing algorithms were found to be consistently competitive with conventional approaches.

After a run completes, the framework produces not a single algorithm but a ranked set of algorithms. In this report, we present results only for the best-performing algorithm of each run. The generated Python code is available as part of the released repository accompanying this paper. Detailed mathematical descriptions of the best-performing algorithms for each task and model are provided in the appendices.[1]

A. Channel Estimation

The PHY layer tasks consist in generating a channel estimator for an orthogonal frequency-division multiplexing (OFDM) multiple-input multiple-output (MIMO) system. We consider two variants of the channel estimation problem, which define the two PHY layer tasks.

In the first variant, referred to as *statistics-agnostic estimation*, no prior knowledge of the channel statistics is assumed. The agent must therefore generate algorithms that operate without access to any second-order channel information. However, the agents are free to make assumptions about the statistics.

In the second variant, referred to as *estimation with known covariance*, the channel estimator is provided with the spatial, frequency, and temporal covariance matrices of the channel. During evaluation, the evaluation tool injects pre-computed covariance matrices for the channel model under test into the agent's workspace. The agent is instructed to generate algorithms that load and exploit these matrices.

[1]These algorithm descriptions were drafted from the generated code by generative AI from the generated code. We have edited and reformatted them only rudimentarily.

In both cases, the channel model, number of transmitted streams, number of receive antennas, pilot pattern, number of subcarriers, and number of OFDM symbols are not disclosed to the agents, in order to encourage the generation of robust and general algorithms. The resulting code must therefore comply with 5G New Radio (NR) physical uplink shared channel (PUSCH) specifications and support arbitrary demodulation reference signal (DMRS) configurations, as well as arbitrary numbers of physical resource blocks (PRBs), transmitted streams, and OFDM symbols. It must also be written in PyTorch and be compatible with Sionna 2.0.

1) Evaluation Setup: The evaluation tool assesses the generated algorithms on an uplink 3GPP urban microcell (UMi) channel model, with four single-antenna user equipments (UEs) and a base station equipped with 16 receive antennas. The OFDM resource grid consists of 72 subcarriers and 14 OFDM symbols, with reference signals placed on the 2nd and 11th symbols. Linear minimum mean square error (LMMSE) equalization and a posteriori probability (APP) demapping are applied after channel estimation, which is not known to the agents.

As the framework requires the evaluation tool to return a single scalar metric, the normalized validation error (NVE) [10] is used. The NVE is defined as

$$\mathrm{NVE} = \frac{1}{N_{\mathrm{snr}}} \sum_{i=1}^{N_{\mathrm{snr}}} \frac{\mathrm{BLER}_{\mathrm{agent}}(\mathrm{SNR}_i)}{\mathrm{BLER}_{\mathrm{pCSI}}(\mathrm{SNR}_i)}, \tag{1}$$

where $\mathrm{BLER}_{\mathrm{agent}}$ and $\mathrm{BLER}_{\mathrm{pCSI}}$ denote the block error rates (BLERs) achieved by the agent's estimator and under perfect channel state information (CSI), respectively, and SNR_i denotes the i-th signal-to-noise ratio (SNR) evaluation point. The BLERs are estimated through Monte Carlo simulation using Sionna with GPU acceleration over an SNR range of -9 to -2 dB. The framework therefore aims to generate algorithms that minimize the NVE: an NVE equal to 1 corresponds to an estimator that achieves the same performance as perfect CSI. Note that an alternative metric would be the mean square error (MSE) between the agent's estimate and the true channel frequency response, but we chose the NVE to account for end-to-end detection performance, which includes equalization, demapping, and channel decoding.

2) Results: Statistics-Agnostic Estimation: To provide some insight into the functioning of the framework, we start by presenting the evolution of the best NVE and success rate across iterations for the statistics-agnostic channel estimation task. Fig. 4 shows the lowest NVE achieved as a function of the iteration index when running the framework with GPT 5.4 and GPT-OSS 120B, together with the success rate, defined as the fraction of generated code that executes without crashing at each iteration. Running the framework over multiple iterations reduces the NVE, as the orchestrator refines ideas and the agents build upon algorithms from previous iterations. However, the success rate is notably higher with the more capable model GPT 5.4. Consequently, a run with GPT 5.4 produces a larger set of working algorithms compared to GPT-OSS 120B.

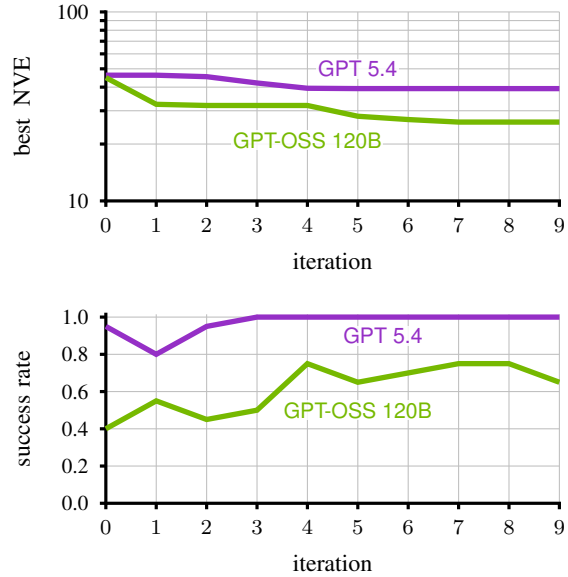
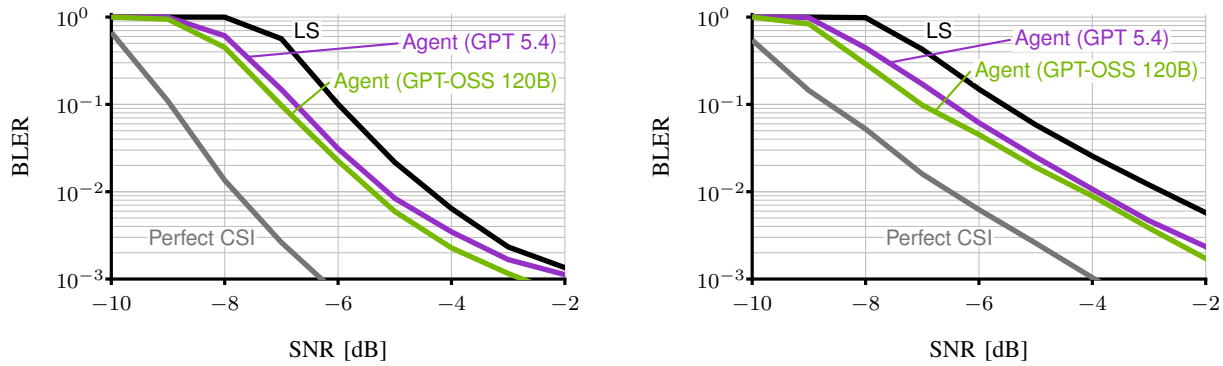


Fig. 4. Evolution of the best NVE and success rate across iterations for the statistics-agnostic channel estimation task

Interestingly, despite the significantly lower success rate of GPT-OSS 120B, the best-performing algorithm generated with this model achieves a lower NVE than the one generated with GPT 5.4 on this task. A possible explanation for this performance gap is that, within the fixed time budget, GPT 5.4 agents performed approximately 22.5% fewer turns, where a turn consists of an LLM call followed by a tool invocation, than GPT-OSS 120B agents, due to the higher per-call latency of the closed-weights model.

We now examine the end-to-end detection performance of the best-performing algorithms. Fig. 5 shows the BLER curves achieved by the best-performing algorithms generated with GPT 5.4 and GPT-OSS 120B, alongside two baselines: perfect CSI and least squares (LS) estimation at pilot positions with linear interpolation. Results are shown for the 3GPP UMi channel used during evaluation, as well as for a 3GPP rural macrocell (RMa) channel that was *not* seen during evaluation, in order to assess generalization. Both algorithms generated by the framework significantly outperform the LS baseline. The algorithm derived with GPT-OSS 120B achieves lower error rates than the one from GPT 5.4 at low SNR, demonstrating that open-weights models can be competitive for algorithm generation.

To gain insight into the generated algorithms, we provide an overview of their functioning. Since the generated code is standard Python, it can be readily inspected, making the resulting algorithms fully explainable and extensible, in contrast to neural network-based approaches. The full code is available in the repository accompanying this paper. Interestingly, the two best-performing algorithms adopt markedly different approaches, illustrating the framework's ability to generate diverse, non-trivial signal processing algorithms. The best-performing algorithm generated with GPT-OSS 120B (see



(a) 3GPP UMi

(b) 3GPP RMa

Fig. 5. BLER curves for the statistics-agnostic channel estimation task

Appendix A-A) refines an initial LS channel estimate by applying orthogonal matching pursuit (OMP) at pilot locations to denoise the estimate through time-domain sparsity enforcement. It then estimates the Doppler correlation and constructs an adaptive time-domain finite impulse response (FIR) filter. A forward Kalman filter combined with a backward Rauch–Tung–Striebel (RTS) smoother tracks the channel evolution across all OFDM symbols. Finally, a second FIR filter and a frequency-domain B-spline filter smooth the reconstructed channel along both time and frequency, yielding a refined channel estimate and its corresponding error variance.

The best-performing algorithm generated with GPT 5.4 (see Appendix A-B) implements an iterative graph-based message-passing channel estimator on the two-dimensional time-frequency resource grid. It treats the grid as a connected graph in which pilot symbols act as anchored observation nodes. The algorithm first creates an initial coarse estimate using a weighted combination of the nearest pilots, then refines it over several iterations by averaging neighboring nodes with dynamic edge weights derived from the phase mismatch between adjacent pilots. It assigns a spatially varying error variance to each node based on its composite distance to the nearest pilots, local pilot quality, and convergence stability.

3) Results: Estimation with Known Covariance: We now turn to the second channel estimation variant, in which agents are provided with the time, frequency, and spatial covariance matrices of the channel. Fig. 6 shows the corresponding BLER curves. As an additional baseline, we include LS estimation with LMMSE interpolation, which applies LMMSE filtering along the time, frequency, and spatial dimensions and therefore also requires knowledge of the corresponding covariance matrices.

For this task, the algorithms generated by GPT-OSS 120B and GPT 5.4 achieve similar error rates, on par with those of the LMMSE baseline. A closer inspection of the generated algorithms reveals them to be specialized variations of the LMMSE filter, confirming the framework's ability to generate non-trivial algorithms but also highlighting a limitation: when a well-known strong solution exists (LMMSE under known covariance), the framework may converge toward a variant of it rather than discovering fundamentally novel approaches.

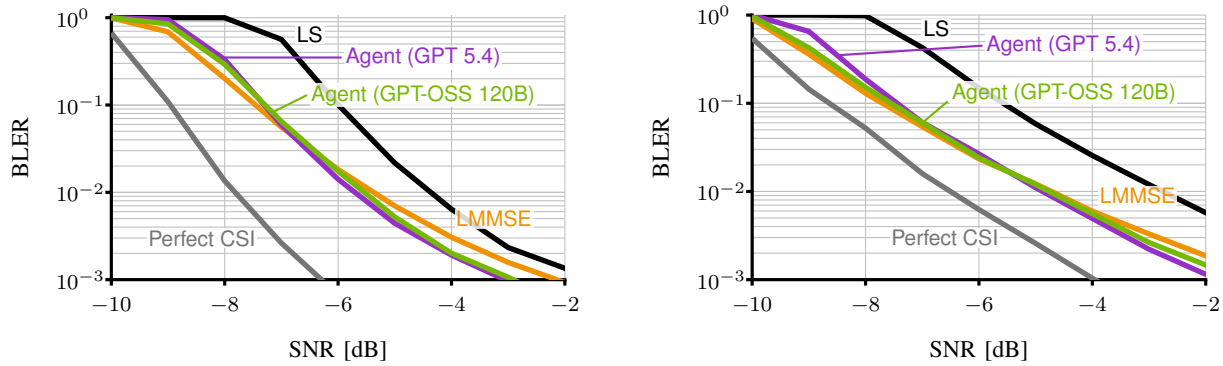
The algorithm generated with GPT-OSS 120B (see Appendix B-A) implements an iterative Kronecker-structured LMMSE estimator. Since the covariance matrices along the time, frequency, and spatial dimensions are provided, their eigendecompositions can be pre-computed. The algorithm projects the noisy LS estimate into the resulting uncorrelated eigendomain and applies a two-pass adaptive Wiener filter iteratively, with a damping factor ensuring stable convergence.

The algorithm generated with GPT 5.4 (see Appendix B-B) implements a separable, sequential LMMSE estimator. It dynamically generates regularized filtering matrices for each dimension based on the current noise variance, sequentially applies them as linear transformations along their respective axes, and constructs the final estimate as a weighted combination of the initial linearly interpolated LS estimate and the intermediate filtered states.

B. Link Adaptation

The MAC layer task consists in implementing a modulation and coding scheme (MCS) selection controller for link adaptation. The controller must maximize spectral efficiency (SE) while maintaining the BLER below a predefined target of 10%. It observes a series of hybrid automatic repeat request (HARQ) acknowledgement/negative acknowledgement (ACK/NACK) feedback together with the corresponding MCS indices, and selects the MCS index for the next transmission. Crucially, the controller has no direct access to the underlying signal-to-interference-plus-noise ratio (SNR) values.

1) Evaluation Setup: The evaluation tool assesses a candidate MCS controller by replaying 50 pre-generated SNR trajectories through a simulated link adaptation loop. The SNR trajectories were generated by simulating UE mobility within an urban scene (Munich) using Sionna RT [11]. For each trajectory, a UE moves along a straight path at a random speed between 3 and 14 m/s, and the channel frequency response is computed at every slot (0.5 ms) via ray tracing. The length of the trajectories was set to 3000 slots, corresponding to



(a) 3GPP UMi

(b) 3GPP RMa

Fig. 6. BLER curves for channel estimation with known covariance

1.5 seconds of simulated time. The per-slot channel gain is obtained by averaging the squared magnitude of the channel frequency response over all subcarriers, yielding the effective SNR per slot.

During evaluation, the simulation replays each SNR trajectory slot by slot. At each slot, the BLER for the current (SNR, MCS) pair is computed using pre-fitted BLER-vs-SNR curves, and a Bernoulli draw determines whether the transmission results in an ACK or a NACK. HARQ feedback is delivered to the MCS selection algorithm in batches of 5 slots to emulate feedback delay. The evaluation tool computes two metrics per scenario: the long-term BLER, which must remain below the target, and the average SE, the optimization objective. The final metric reported to the agent is the SE averaged across all 50 scenarios, and the evaluation is marked as successful only if the BLER constraint is satisfied on every scenario. As additional information, the tool reports BLER statistics across all trajectories, as well as the number of trajectories that did not meet the BLER constraint.

2) Results: As baseline, we consider outer loop link adaptation (OLLA) [12] for MCS selection. For fairness, the baseline hyperparameter that controls the SNR back-off step size applied after each failed transmission was fine-tuned on the trajectories used by the evaluation tool to maximize the SE while satisfying the BLER constraint on all sequences. Table I shows the benchmarking results, computed on a separate non-overlapping set of 50 trajectories also generated with Sionna RT on the Munich scene, to ensure that the selected algorithms have not overfit to the evaluation set. All approaches satisfy the BLER constraint on all 50 sequences. The algorithm generated with GPT-OSS 120B achieves an SE slightly lower than the fine-tuned OLLA baseline, while the algorithm generated with GPT 5.4 achieves a significant SE gain of more than 3%, showing that agentic AI can generate MAC layer algorithms that are competitive with conventional approaches.

To gain insight into the generated link adaptation algorithms, we provide an overview of their functioning. The best-performing algorithm generated with GPT-OSS 120B (see Appendix C-A) implements a Monte Carlo look-ahead link adaptation strategy based on a one-dimensional particle filter. It tracks the posterior distribution of the unobservable SNR using a set of weighted particles that evolve via a random walk and are updated using binary ACK/NACK feedback. To select the MCS, it evaluates the expected immediate and one-step-ahead SE for all candidate MCS values across the particle distribution.

TABLE I
LINK ADAPTATION BENCHMARKING RESULTS ON THE HELD-OUT SET. SE DENOTES SPECTRAL EFFICIENCY.

Algorithm	Avg. SE (bps/Hz)	Success	vs. Baseline
GPT-OSS 120B	3.3976	50/50	−0.64%
GPT 5.4	3.5370	50/50	+3.43%
OLLA baseline	3.4196	50/50	—

The best-performing algorithm generated with GPT 5.4 (see Appendix C-B) is simpler. It implements a grid-based discrete Markov filter paired with an asymmetric OLLA mechanism. It discretizes the SNR space into a fixed grid and tracks the probability distribution of the true SNR over time using Gaussian convolution for time evolution and Bayesian updates based on ACK/NACK feedback. A trimmed mean of the posterior distribution yields an SNR estimate, which is then adjusted by an asymmetric OLLA offset, stepping up for ACKs and stepping down more aggressively for NACKs, before selecting the final MCS.

IV. Conclusion

This paper explores the ability of agentic AI to autonomously design wireless communication algorithms. To that end, we built a dedicated framework and evaluated it on three tasks spanning both the PHY and MAC layers. Using off-the-shelf LLMs, the framework generates competitive algorithms in a matter of hours: the best-performing algorithms significantly outperform simple baselines, match strong classical methods such as LMMSE channel estimation, and in the case of link adaptation, surpass a fine-tuned OLLA baseline by more than 3% in spectral efficiency.

A noteworthy property of the agentic approach is that the generated algorithms are conventional signal processing code, fully explainable and extensible. This stands in contrast to neural network methods whose internal workings are difficult to interpret.

However, these early experiments also reveal a limitation of current agentic AI systems: when a strong classical solution exists, the framework tends to converge toward variations of it rather than discovering fundamentally novel approaches. It remains open for future work to investigate whether this lack of creativity is an inherent limitation of LLM-based research or whether our choice of experimental problems did not allow for significant improvements over the state of the art. We note that the framework is at an early stage of development, and many improvements are possible. Advancing agentic AI to enable the discovery of genuinely novel wireless communication algorithms is a promising and challenging direction, as this form of algorithmic design may become common practice.

References

[1] Z. Li, Z. Li, Z. Guo, X. Ren, and C. Huang, "DeepCode: Open agentic coding," *arXiv preprint arXiv:2512.07921*, 2025.

[2] A. Elovic, "GPT Researcher," https://github.com/assafelovic/gpt-researcher, 2023.

[3] J. Hoydis, S. Cammerer, F. Aït Aoudia, A. Vem, N. Binder, G. Marcus, and A. Keller, "Sionna: An open-source library for next-generation physical layer research," *arXiv preprint arXiv:2203.11854*, 2022.

[4] R. T. Lange, Y. Imajuku, and E. Cetin, "ShinkaEvolve: Towards open-ended and sample-efficient program evolution," *arXiv preprint arXiv:2509.19349*, 2025.

[5] A. Sharma, "OpenEvolve: An open-source evolutionary coding agent," https://github.com/algorithmicsuperintelligence/openevolve, 2025.

[6] E. Hemberg, S. Moskal, and U.-M. O'Reilly, "Evolving code with a large language model," *Genetic Programming and Evolvable Machines*, vol. 25, no. 2, p. 21, 2024.

[7] F. Liu, R. Zhang, Z. Xie, R. Sun, K. Li, X. Lin, Z. Wang, Z. Lu, and Q. Zhang, "LLM4AD: A platform for algorithm design with large language model," *arXiv preprint arXiv:2412.17287*, 2024.

[8] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian *et al.*, "AlphaEvolve: A coding agent for scientific and algorithmic discovery," *arXiv preprint arXiv:2506.13131*, 2025.

[9] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The Faiss library," *IEEE Transactions on Big Data*, 2025.

[10] T. Gruber, S. Cammerer, J. Hoydis, and S. ten Brink, "On deep learning-based channel decoding," in *Proc. 51st Annual Conf. Inform. Sciences and Systems (CISS)*, 2017, pp. 1–6.

[11] F. Aït Aoudia, J. Hoydis, M. Nimier-David, B. Nicolet, S. Cammerer, and A. Keller, "Sionna RT: Technical report," *arXiv preprint arXiv:2504.21719*, 2025.

[12] K. I. Pedersen, G. Monghal, I. Z. Kovacs, T. E. Kolding, A. Pokhariyal, F. Frederiksen, and P. Mogensen, "Frequency Domain Scheduling for OFDMA with Limited and Noisy Channel Feedback," in *IEEE Vehicular Technology Conference (VTC)*, 2007, pp. 1792–1796.

Appendix A
Statistics-Agnostic Channel Estimation: Generated Algorithms

A. GPT-OSS 120B

This algorithm implements a channel estimation and tracking pipeline for OFDM-based systems. It refines an initial noisy LS channel estimate through a sequence of signal processing techniques designed to exploit both the sparse nature of multipath channels and their temporal-frequency correlations. The core functioning consists of applying OMP at pilot locations to denoise the estimates by enforcing time-domain sparsity. It then estimates the Doppler correlation to generate an adaptive time-domain FIR filter. A forward Kalman filter combined with a backward RTS smoother tracks the channel evolution across all OFDM symbols (both pilots and data). Finally, a second time-domain FIR filter and a frequency-domain B-spline filter are applied to smooth the reconstructed channel across both time and frequency axes, yielding a highly refined channel estimate and its corresponding error variance.

Let M be the number of subcarriers (FFT size) and T be the total number of OFDM symbols. The algorithm relies on the following predefined hyperparameters: $K = 4$ for the OMP sparsity level, $L_{\text{max}} = 16$ for the maximum channel tap delay, and $L_{\text{FIR}} = 7$ for the length of the time-domain FIR kernel.

1) Initial LS Estimation: The algorithm begins with a baseline LS channel estimator. Let the initial channel estimates over time t and frequency f be $\hat{H}_{\text{LS}}(t, f)$ and the corresponding error variance be $\sigma^2_{\text{LS}}(t, f)$. Let $\mathcal{P}$ denote the set of time indices t that contain pilot symbols.

2) OMP Sparsification: For the time indices $t \in \mathcal{P}$, the algorithm suppresses noise by selecting the K most dominant time-domain taps.

- Dictionary Construction: A discrete Fourier transform (DFT) dictionary $A \in \mathbb{C}^{M \times L_{\text{max}}}$ is built. The elements are defined as
$$A_{m,l} = \exp\left(-j\frac{2\pi ml}{M}\right),$$
where $m \in \{0, \ldots, M-1\}$ and $l \in \{0, \ldots, L_{\text{max}} - 1\}$.
- Correlation & Support Selection: The correlation $c(t, l)$ is computed
$$c(t, l) = \sum_{m=0}^{M-1} \hat{H}_{\text{LS}}(t, m) A^*_{m,l}.$$
The support set $\Omega(t)$ of size K is formed by taking the indices l that yield the largest magnitude $|c(t, l)|$.
- Least Squares on Support: Let $A_{\Omega(t)} \in \mathbb{C}^{M \times K}$ be the sub-matrix of A. The sparse time-domain taps $\hat{x}(t) \in \mathbb{C}^K$ are estimated via
$$\hat{x}(t) = (A^H_{\Omega(t)} A_{\Omega(t)})^{-1} A^H_{\Omega(t)} \hat{H}_{\text{LS}}(t).$$
- Reconstruction:
$$\hat{H}_{\text{OMP}}(t) = A_{\Omega(t)} \hat{x}(t)$$
$$E_{\text{OMP}}(t, f) = |\hat{H}_{\text{LS}}(t, f) - \hat{H}_{\text{OMP}}(t, f)|^2$$

3) Doppler Coefficient Estimation: The temporal correlation a_{est} is calculated using the OMP estimates
$$a_{\text{est}} = \text{clamp}\left(\frac{\mathbb{E}_{t,f}[\Re(\hat{H}_{\text{OMP}}(t, f)\, \hat{H}^*_{\text{OMP}}(t+1, f))]}{\mathbb{E}_{t,f}[|\hat{H}_{\text{OMP}}(t, f)|^2] + \epsilon}, 0.5, 0.99\right)$$
where $\epsilon = 10^{-12}$ is a small constant added to the denominator to prevent division by zero, and $\text{clamp}(x, a, b)$ clamps x to the interval $[a, b]$.

4) Time-Domain FIR Filtering (Pilots): A symmetric FIR kernel $w_t(\tau)$ of length L_{FIR} is constructed as

$$w_t(\tau) = \frac{a_{\text{est}}^{|\tau|}}{\sum_i a_{\text{est}}^{|i|}}.$$

This filter is applied via convolution over the time dimension,

$$\tilde{H}_{\text{pilot}}(t,f) = (\hat{H}_{\text{OMP}} * w_t)(t,f),$$
$$\tilde{E}_{\text{pilot}}(t,f) = (E_{\text{OMP}} * w_t)(t,f).$$

5) Forward Kalman Filter: A scalar Kalman filter tracks the channel across all OFDM symbols $t \in \{0,\ldots,T-1\}$. Let a_k be the state transition coefficient of a first-order Autoregressive (AR(1)) model used as a generic prior (set to 0.99), and let $q = 1 - a_k^2$ denote the corresponding process noise variance.

- Initialization ($t = 0$): If $0 \in \mathcal{P}$, $\hat{H}_{\text{fwd}}(0) = \tilde{H}_{\text{pilot}}(0)$ and $P_{\text{fwd}}(0) = \tilde{E}_{\text{pilot}}(0)$. Otherwise, initialized with $\hat{H}_{\text{LS}}(0)$ and $\sigma^2_{\text{LS}}(0)$.
- Prediction Step ($t > 0$):

$$H_{\text{pred}}(t) = a_k \hat{H}_{\text{fwd}}(t-1),$$
$$P_{\text{pred}}(t) = a_k^2 P_{\text{fwd}}(t-1) + q$$

- Measurement Update Step ($t \in \mathcal{P}$):

$$K_{\text{gain}}(t) = \frac{P_{\text{pred}}(t)}{P_{\text{pred}}(t) + \tilde{E}_{\text{pilot}}(t)},$$
$$\hat{H}_{\text{fwd}}(t) = H_{\text{pred}}(t) + K_{\text{gain}}(t)\left(\tilde{H}_{\text{pilot}}(t) - H_{\text{pred}}(t)\right),$$
$$P_{\text{fwd}}(t) = (1 - K_{\text{gain}}(t))P_{\text{pred}}(t).$$

If $t \notin \mathcal{P}$, $\hat{H}_{\text{fwd}}(t) = H_{\text{pred}}(t)$ and $P_{\text{fwd}}(t) = P_{\text{pred}}(t)$.

6) Backward RTS Smoother:

- Initialization:

$$\hat{H}_{\text{sm}}(T-1) = \hat{H}_{\text{fwd}}(T-1),$$
$$P_{\text{sm}}(T-1) = P_{\text{fwd}}(T-1).$$

- Smoothing Step:

$$C(t) = \frac{P_{\text{fwd}}(t) a_k}{P_{\text{pred}}(t+1)},$$
$$\hat{H}_{\text{sm}}(t) = \hat{H}_{\text{fwd}}(t) + C(t)\left(\hat{H}_{\text{sm}}(t+1) - H_{\text{pred}}(t+1)\right),$$
$$P_{\text{sm}}(t) = P_{\text{fwd}}(t) + C(t)^2\left(P_{\text{sm}}(t+1) - P_{\text{pred}}(t+1)\right).$$

7) Final Time and Frequency Smoothing:

- Time FIR Filter: The adaptive kernel $w_t(\tau)$ is applied across the entire time axis t,

$$\hat{H}_{\text{time}}(t,f) = (\hat{H}_{\text{sm}} * w_t)(t,f),$$
$$P_{\text{time}}(t,f) = (P_{\text{sm}} * w_t)(t,f).$$

- Frequency Spline Filter: A fixed 1D discrete convolution kernel approximating a cubic B-spline, defined as $w_f = [0.125, 0.375, 0.5, 0.375, 0.125]$, is applied across the frequency axis f,

$$\hat{H}_{\text{final}}(t,f) = (\hat{H}_{\text{time}} * w_f)(t,f),$$
$$P_{\text{final}}(t,f) = (P_{\text{time}} * w_f)(t,f).$$

B. GPT 5.4

This algorithm implements an iterative, graph-based message-passing channel estimator on the 2D time-frequency resource grid. It interpolates missing channel estimates by treating the resource grid as a connected graph where pilot symbols act as anchored observation nodes. The algorithm initializes a coarse estimate using a weighted combination of the nearest pilots. It then refines this state over several iterations by averaging neighboring nodes, utilizing dynamic edge weights derived from the phase mismatch between adjacent pilots. Finally, it assigns a spatially varying error variance to each node based on its composite distance to the nearest pilots, local pilot quality, and solver convergence stability.

Let N_{t} be the number of OFDM symbols (time dimension) and N_{f} be the number of subcarriers (frequency dimension). Let (t,f) denote the discrete coordinates on this grid, where $t \in \{0,\ldots,N_{\text{t}}-1\}$ and $f \in \{0,\ldots,N_{\text{f}}-1\}$.

Let $\hat{H}_{\text{LS}}(t,f)$ be the initial LS channel estimate, $\sigma^2_{\text{LS}}(t,f)$ be the corresponding LS error variance, and N_0 be the scalar noise variance. Let $M(t,f) \in \{0,1\}$ be the binary pilot mask, where 1 indicates a pilot location and 0 indicates a data location.

1) Distance Mapping and Pilot Support Density: The algorithm computes spatial metrics relative to the pilot positions to dictate regional smoothing behavior.

- Distances: Let $d_{\text{t}}(t,f)$ and $d_{\text{f}}(t,f)$ be the absolute distance (in indices) from coordinate (t,f) to the nearest pilot in the time and frequency axes, respectively. A composite distance score $d_{\text{score}}(t,f)$ is defined as

$$d_{\text{score}}(t,f) = d_{\text{f}}(t,f) + 1.55\, d_{\text{t}}(t,f).$$

- Region Partitioning: Non-pilot elements ($M(t,f) = 0$) are partitioned into three mutually exclusive regions based on $d_{\text{score}}(t,f)$:
  - Near Region: $d_{\text{score}}(t,f) \le 1.7$
  - Mid Region: $1.7 < d_{\text{score}}(t,f) \le 3.55$
  - Far Region: $d_{\text{score}}(t,f) > 3.55$
- Pilot Support Density (S): The local pilot density $S(t,f)$ is initialized as the float representation of the mask: $S^{(0)}(t,f) = M(t,f)$. It is updated over $k \in \{1,2\}$ iterations via a spatial blur,

$$\begin{aligned} S^{(k)}(t,f) = {} & 0.30\, S^{(k-1)}(t,f) \\ & + 0.27\big(S^{(k-1)}(t,f-1) + S^{(k-1)}(t,f+1)\big) \\ & + 0.08\big(S^{(k-1)}(t-1,f) + S^{(k-1)}(t+1,f)\big). \end{aligned}$$

The final support is clamped,

$$S(t,f) = \max(S^{(2)}(t,f), 10^{-3}).$$

2) Initial State Construction: A starting channel state $H^{(0)}(t,f)$ is generated for the iterative solver.

- Let $H_{\text{t}}(t,f)$ and $H_{\text{f}}(t,f)$ be the LS channel values of the temporally and frequency nearest pilots to (t,f), respectively.

- A base blend is initialized:

$$H_{\text{base}}^{(0)}(t,f) = 0.66\, H_{\text{f}}(t,f) + 0.34\, H_{\text{t}}(t,f)$$

- The base blend undergoes $k \in \{1,2\}$ passes of a cross-shaped smoothing filter:

$$\begin{aligned} H_{\text{base}}^{(k)}(t,f) &= 0.28\, H_{\text{base}}^{(k-1)}(t,f) \\ &+ 0.31\big(H_{\text{base}}^{(k-1)}(t,f-1) + H_{\text{base}}^{(k-1)}(t,f+1)\big) \\ &+ 0.05\big(H_{\text{base}}^{(k-1)}(t-1,f) + H_{\text{base}}^{(k-1)}(t+1,f)\big) \end{aligned}$$

- Pilot Anchor: The initial state $H^{(0)}(t,f)$ strictly anchors pilot locations to their true LS values:

$$H^{(0)}(t,f) = \begin{cases} \hat{H}_{\text{LS}}(t,f), & M(t,f) = 1 \\ H_{\text{base}}^{(2)}(t,f), & M(t,f) = 0 \end{cases}$$

3) Edge Weight Computation (Phase Mismatch): Dynamic transition weights (gates) are computed to preserve channel edges. Let the function $\text{clamp}(x,a,b)$ denote restricting the value of x to the interval $[a,b]$.

- Phase Mismatch Function: For any two adjacent complex estimates a and b, the phase correlation is $c = ab^*$. The unit phase vector is $\phi = \frac{c}{\max(|c|,10^{-4})}$. The mismatch $m(a,b)$ is:

$$m(a,b) = \frac{|a - \phi b|}{0.5\max(|a|+|b|,\ 2\cdot 10^{-3})}$$

- Let $m_{\text{t}}(t,f) = m(\hat{H}_{\text{LS}}(t,f), \hat{H}_{\text{LS}}(t-1,f))$ and $m_{\text{f}}(t,f) = m(\hat{H}_{\text{LS}}(t,f), \hat{H}_{\text{LS}}(t,f-1))$ be the mismatches. These are strictly evaluated only if both nodes in the pair are valid pilots; otherwise, $m_{\text{t}}(t,f) = 0$ and $m_{\text{f}}(t,f) = 0$.
- Transmission Gates: The directional gates $g_{\text{t}}(t,f)$ and $g_{\text{f}}(t,f)$ are defined as:

$$\begin{aligned} g_{\text{t}}(t,f) &= \text{clamp}(1.0 - 0.08\, m_{\text{t}}(t,f),\ 0.92,\ 0.985) \\ g_{\text{f}}(t,f) &= \text{clamp}(1.0 - 0.14\, m_{\text{f}}(t,f),\ 0.94,\ 0.997) \end{aligned}$$

4) Iterative Graph Smoothing: The channel $H^{(i)}(t,f)$ is updated as follows (for $i \in \{0,\ldots,6\}$):

- Static Weights: The observation and directional transition weights are constant across iterations,

$$\begin{aligned} W_{\text{obs}}(t,f) &= M(t,f) \\ &\quad \cdot \text{clamp}\left(0.16 + \frac{0.56}{\max(N_0, 10^{-4})},\ 0,\ 8.0\right) \\ W_L &= 1.235 \cdot g_{\text{f}}(t,f), \\ W_R &= 1.235 \cdot g_{\text{f}}(t,f+1), \\ W_U &= 0.055 \cdot g_{\text{t}}(t,f), \\ W_D &= 0.055 \cdot g_{\text{t}}(t+1,f) \end{aligned}$$

- Update Rule: At each iteration i, a new unblended state $H_{\text{new}}(t,f)$ is computed:

$$\begin{aligned} H_{\text{new}} &= \frac{W_{\text{obs}}\hat{H}_{\text{LS}} + W_L H_{\text{left}}^{(i)} + W_R H_{\text{right}}^{(i)}}{W_{\text{obs}} + W_L + W_R + W_U + W_D + 10^{-6}} \\ &\quad + \frac{W_U H_{\text{up}}^{(i)} + W_D H_{\text{down}}^{(i)}}{W_{\text{obs}} + W_L + W_R + W_U + W_D + 10^{-6}}, \end{aligned}$$

where

$$\begin{aligned} H_{\text{left}}^{(i)} &= H^{(i)}(t,f-1), \\ H_{\text{right}}^{(i)} &= H^{(i)}(t,f+1), \\ H_{\text{up}}^{(i)} &= H^{(i)}(t-1,f), \\ H_{\text{down}}^{(i)} &= H^{(i)}(t+1,f). \end{aligned}$$

- Let the momentum factor be $\beta^{(i)} = 0.972$ for $i < 5$ and $\beta^{(i)} = 0.982$ for $i \geq 5$. The state is updated via a momentum blend and re-anchored to the pilots:

$$H^{(i+1)} = \begin{cases} \hat{H}_{\text{LS}}(t,f), & M = 1 \\ \beta^{(i)} H_{\text{new}} + (1-\beta^{(i)}) H^{(i)}, & M = 0 \end{cases},$$

where the (t,f) arguments are omitted for brevity.

5) Final Frequency Smoothing: After 7 iterations, a localized smoothing pass is applied strictly along the frequency axis to the final state $H^{(7)}(t,f)$:

$$\begin{aligned} H_{\text{freq}}(t,f) &= 0.16\, H^{(7)}(t,f) \\ &\quad + 0.42\, H^{(7)}(t,f-1) + 0.42\, H^{(7)}(t,f+1). \end{aligned}$$

The output channel estimate $\hat{H}_{final}(t,f)$ is $H_{\text{freq}}(t,f)$ for data nodes and $\hat{H}_{\text{LS}}(t,f)$ for pilot nodes.

6) Error Variance Assignment: The final error variance $E_{\text{var}}(t,f)$ is constructed regionally based on local metrics:

- Let MSE_{p} be the scalar mean of $|\hat{H}_{\text{LS}}(t,f) - H^{(7)}(t,f)|^2$ evaluated strictly over all pilot nodes. Let $\bar{e}_0$ be the scalar global mean of the LS variance $\sigma_{\text{LS}}^2(t,f)$. Let $H^{(6)}(t,f)$ and $H^{(7)}(t,f)$ denote the channel states immediately before and after the final (seventh) message-passing iteration, respectively.
- Local Metrics:

$$C(t,f) = \text{clamp}\left(\frac{|H^{(7)}(t,f) - H^{(6)}(t,f)|}{\max(|H^{(7)}(t,f)|, 3\cdot 10^{-3})},\ 0.0,\ 0.5\right)$$

$$S_{\text{exc}}(t,f) = \max\left(\frac{1}{\max(S(t,f), 0.28)} - 1,\ 0\right)$$

$$Q_{\text{p}} = \text{clamp}\left(\frac{\text{MSE}_{\text{p}}}{0.25\bar{e}_0 + 10^{-4}},\ 0,\ 4\right)$$

- Distance Penalties:

$$\begin{aligned} P_{\text{near}} &= 0.0013 + 0.0028\, d_{\text{score}} + 0.0018\, S_{\text{exc}} \\ P_{\text{mid}} &= 0.0048 + 0.0055\, d_{\text{score}} + 0.0042\, S_{\text{exc}} \\ P_{\text{far}} &= 0.0092 + 0.0080\, d_{\text{score}} + 0.0070\, S_{\text{exc}} \\ &\quad + 0.0038\, Q_{\text{p}} \end{aligned}$$

- Distance Penalties:

$$P_{\text{near}} = 0.0013 + 0.0028\, d_{\text{score}} + 0.0018\, S_{\text{exc}}$$
$$P_{\text{mid}} = 0.0048 + 0.0055\, d_{\text{score}} + 0.0042\, S_{\text{exc}}$$
$$P_{\text{far}} = 0.0092 + 0.0080\, d_{\text{score}} + 0.0070\, S_{\text{exc}} + 0.0038\, Q_{\text{p}}$$

- Final Variance Equations: For the nodes for which $M(t,f) = 1$, the equations are

$$E_{\text{var}} = \max(0.26\, \sigma_{\text{LS}}^2 + 0.006\, N_0,\ 10^{-4}),$$

and for non-pilot nodes, depending on their region, the equations are

$$E_{\text{near}} = 0.070\, \sigma_{\text{LS}}^2 + 0.060\, \text{MSE}_{\text{p}} + P_{\text{near}} + 0.0095\, C,$$
$$E_{\text{mid}} = 0.092\, \sigma_{\text{LS}}^2 + 0.235\, \text{MSE}_{\text{p}} + P_{\text{mid}} + 0.0160\, C,$$
$$E_{\text{far}} = 0.125\, \sigma_{\text{LS}}^2 + 0.545\, \text{MSE}_{\text{p}} + P_{\text{far}} + 0.0240\, C,$$

where all quantities depend on (t,f). All values are finally clamped to a minimum numerical floor of 10^{-4}.

Appendix B
Channel Estimation with Known Covariance: Generated Algorithms

A. GPT-OSS 120B

This algorithm implements an iterative Kronecker-structured LMMSE channel estimator. By pre-computing the eigen-decompositions of the time, frequency, and space covariance matrices, the algorithm efficiently projects a baseline noisy LS channel estimate into an uncorrelated eigen-domain. It then applies a two-pass adaptive Wiener filter iteratively. The first pass filters the signal in the eigen-domain using the global observation variance, while the second pass applies a localized correction in the original element domain. A damping factor ensures stable convergence across iterations, ultimately producing a highly refined channel estimate and its corresponding error variance.

Let N_{s}, N_{t}, and N_{f} be the number of spatial antennas, time OFDM symbols, and frequency subcarriers, respectively. Let $H_{\text{LS}}(s,t,f)$ be the initial Least Squares channel estimate and $V_{\text{LS}}(s,t,f)$ be its corresponding error variance.

1) Offline Pre-computation (Eigen-Decomposition): The algorithm assumes the full channel covariance matrix can be separated into three smaller Hermitian covariance matrices: R_{s} (space), R_{t} (time), and R_{f} (frequency).

- Eigen-decomposition: Each matrix is decomposed into its real eigenvalues and complex eigenvectors (U^H denotes the conjugate transpose):

$$R_{\text{s}} = U_{\text{s}}\, \text{diag}(\Lambda_{\text{s}}) U_{\text{s}}^H$$
$$R_{\text{t}} = U_{\text{t}}\, \text{diag}(\Lambda_{\text{t}}) U_{\text{t}}^H$$
$$R_{\text{f}} = U_{\text{f}}\, \text{diag}(\Lambda_{\text{f}}) U_{\text{f}}^H$$

- Prior Eigenvalue Product: The combined 3D eigenvalue tensor is formed via the outer product:

$$\Lambda_{\text{prod}}(s,t,f) = \Lambda_{\text{s}}[s] \cdot \Lambda_{\text{t}}[t] \cdot \Lambda_{\text{f}}[f]$$

- Marginal and Element Variances: The diagonal of each covariance matrix represents the marginal variance, computed using the absolute squared eigenvectors:

$$\text{var}_{\text{s}} = |U_{\text{s}}|^2 \Lambda_{\text{s}},$$
$$\text{var}_{\text{t}} = |U_{\text{t}}|^2 \Lambda_{\text{t}},$$
$$\text{var}_{\text{f}} = |U_{\text{f}}|^2 \Lambda_{\text{f}}$$

where $|U|^2$ denotes the element-wise squared amplitude of U. The combined prior variance for each element in the original domain is:

$$\text{Var}_{\text{elem}}(s,t,f) = \text{var}_{\text{s}}[s] \cdot \text{var}_{\text{t}}[t] \cdot \text{var}_{\text{f}}[f]$$

2) Initial Variance Transformation: Before filtering, the element-domain LS error variance V_{LS} is projected into the real-valued eigen-domain using tensor contractions to initialize $V_{\text{eig}}^{(0)}$:

$$V_{\text{eig}}^{(0)} = |U_{\text{s}}|^2 |U_{\text{t}}|^2 |U_{\text{f}}|^2 V_{\text{LS}}$$

3) Iterative Wiener Filtering: Let ϵ be a small machine-precision constant added for numerical stability, $d \in (0,1]$ be the damping factor, and τ_{tol} be the relative convergence tolerance. The channel state is initialized as $H^{(0)} = H_{\text{LS}}$. The algorithm loops over iterations $i = 0, 1, \ldots$ through the following steps until the relative mean absolute change between $H^{(i)}$ and $H_{\text{ref}'}^{(i)}$ is less than τ_{tol} or the maximum number of iterations is reached:

- Transform to Eigen-domain:

$$H_{\text{eig}}^{(i)} = U_{\text{s}}^H U_{\text{t}}^H U_{\text{f}}^H H^{(i)}$$

- First Wiener Pass (Eigen-domain):

$$W_1^{(i)} = \frac{\Lambda_{\text{prod}}}{\Lambda_{\text{prod}} + V_{\text{eig}}^{(i)}},$$
$$H_{\text{eig}'}^{(i)} = H_{\text{eig}}^{(i)} \cdot W_1^{(i)},$$
$$V_{\text{post}}^{(i)} = \frac{\Lambda_{\text{prod}} \cdot V_{\text{eig}}^{(i)}}{\Lambda_{\text{prod}} + V_{\text{eig}}^{(i)}}.$$

- Transform to Element-domain:

$$H_{\text{ref}}^{(i)} = U_{\text{s}} U_{\text{t}} U_{\text{f}} H_{\text{eig}'}^{(i)},$$
$$V_{\text{post}'}^{(i)} = |U_{\text{s}}|^2 |U_{\text{t}}|^2 |U_{\text{f}}|^2 V_{\text{post}}^{(i)}.$$

- Second Wiener Pass (Element-domain):

$$W_2^{(i)} = \frac{\text{Var}_{\text{elem}}}{\text{Var}_{\text{elem}} + V_{\text{post}'}^{(i)} + \epsilon},$$
$$H_{\text{ref}'}^{(i)} = H_{\text{ref}}^{(i)} \cdot W_2^{(i)}.$$

- State and Variance Update:

$$V_{\text{final}}^{(i)} = V_{\text{post}}^{(i)} \cdot \left(\frac{\Lambda_{\text{prod}}}{\Lambda_{\text{prod}} + V_{\text{post}}^{(i)} + \epsilon} \right)$$

Using the damping factor d, the states are updated for the next iteration:

$$H^{(i+1)} = d \cdot H_{\text{ref}'}^{(i)} + (1-d) \cdot H^{(i)},$$
$$V_{\text{eig}}^{(i+1)} = d \cdot V_{\text{final}}^{(i)} + (1-d) \cdot V_{\text{eig}}^{(i)}.$$

4) Final Output: Let I denote the final iteration index upon termination. The final eigen-domain variance $V_{\text{final}}^{(I)}$ is projected back into the element domain:

$$V_{\text{final}'} = |U_{\text{s}}|^2 |U_{\text{t}}|^2 |U_{\text{f}}|^2 V_{\text{final}}^{(I)}.$$

The algorithm returns $H^{(I)}$ and $V_{\text{final}'}$.

B. GPT 5.4

This algorithm implements a separable, sequential LMMSE channel estimator. Instead of computing a computationally prohibitive joint filtering matrix across all dimensions, it utilizes independent, pre-computed covariance matrices for the frequency, time, and spatial domains. The algorithm dynamically generates regularized filtering matrices for each domain based on the current noise variance. These filters are then sequentially applied as linear transformations along their respective axes of the initial noisy LS channel estimate. To preserve edge details and avoid over-smoothing, the final channel estimate is constructed by taking a weighted linear combination of the initial LS estimate and the intermediate filtered states.

Let N_0 be the scalar noise variance representing the mean noise level (computed as the mean of the real part of the input noise tensor). Let $\hat{H}_{\text{LS}}$ be the initial LS channel estimate and σ_{LS}^2 be its corresponding error variance tensor. Let R_{f}, R_{t}, and R_{s} be the pre-computed covariance matrices for the frequency, time, and space (receive antenna) dimensions, respectively.

1) Filter Regularization Parameters: The algorithm computes an axis-specific regularization parameter α (acting as an effective noise-to-signal ratio) that scales linearly with the input noise variance N_0:

$$\alpha_{\text{f}} = 0.0077 + 0.094 N_0$$
$$\alpha_{\text{t}} = 0.0057 + 0.053 N_0$$
$$\alpha_{\text{s}} = 0.0027 + 0.021 N_0$$

2) Covariance Normalization and Filter Construction: For each covariance matrix $C \in \{R_{\text{f}}, R_{\text{t}}, R_{\text{s}}\}$ and its corresponding parameter α, a specific filtering matrix W is generated.

First, the parameter α is clamped to a numerical floor: $\tilde{\alpha} = \max(\alpha, 10^{-6})$.

Next, the covariance matrix is trace-normalized by its mean real diagonal element to ensure consistent scaling. Let N be the dimension of the square matrix C:

$$\tau = \max\left(\frac{1}{N}\sum_{i=1}^{N} C_{i,i}, 10^{-6}\right)$$
$$\tilde{C} = \frac{1}{\tau} C$$

Then, the standard LMMSE filter matrix W is constructed using the normalized covariance and the clamped regularization parameter, where I is the identity matrix:

$$W = \tilde{C}(\tilde{C} + \tilde{\alpha} I)^{-1}$$

This yields the three filtering matrices W_{f}, W_{t}, and W_{s}.

3) Sequential Axis Filtering: The filters are applied sequentially to the channel estimate tensor. Each filtering operation acts strictly along its targeted dimension. Let $\times_{\text{d}}$ denote the tensor mode-product (matrix multiplication along a specific axis).

- Frequency Filtering:
$$H_{\text{f}} = W_{\text{f}} \times_{\text{f}} \hat{H}_{\text{LS}}$$
- Time Filtering:
$$H_{\text{t}} = W_{\text{t}} \times_{\text{t}} H_{\text{f}}$$
- Spatial Filtering:
$$H_{\text{s}} = W_{\text{s}} \times_{\text{s}} H_{\text{t}}$$

4) Final State Blending and Variance Estimation: The algorithm generates a composite final channel estimate H_{final} by blending the original observation with the progressively smoothed intermediate states:

$$H_{\text{final}} = 0.08\hat{H}_{\text{LS}} + 0.12 H_{\text{f}} + 0.19 H_{\text{t}} + 0.61 H_{\text{s}}$$

Finally, the output error variance E_{final} is modeled as a statically scaled version of the initial LS error variance, clamped to a numerical floor to maintain stability:

$$E_{\text{final}} = \max(0.553 \cdot \sigma_{\text{LS}}^2, 10^{-6})$$

Appendix C
Link Adaptation: Generated Algorithms

A. GPT-OSS 120B

This algorithm implements a Monte Carlo look-ahead link adaptation strategy based on a 1D Particle Filter. It tracks the posterior distribution of the unobservable SNR using a set of weighted particles that evolve via a random walk and are updated using binary ACK/NACK feedback. To select the optimal MCS, it evaluates the expected immediate and one-step future spectral efficiency for all candidate MCS values across the particle distribution, ensuring the average BLER remains below a specified target.

Let $N_{\text{particles}}$ be the total number of particles (set to 100). Let $s_i(t)$ denote the SNR hypothesis of particle i at time step t, and $w_i(t)$ be its corresponding probability weight. Let T_{BLER} be the target BLER. Let $\epsilon = 10^{-12}$ be a small constant for numerical stability. Let $p_{\text{cons}} = 0.2$ be the conservative percentile and $\Delta_{\text{margin}} = 0.5$ dB be the safety margin.

1) Initialization: If no history exists, the particles are initialized uniformly across the allowed SNR range $[\text{SNR}_{\text{min}}, \text{SNR}_{\text{max}}]$. All weights are initialized equally:

$$w_i(0) = \frac{1}{N_{\text{particles}}}$$

2) State Prediction (Random Walk): For each new observation step, the hidden SNR is assumed to evolve according to a Gaussian random walk:

$$s_i(t) = s_i(t-1) + n_i$$

where n_i is process noise drawn from a normal distribution $n_i \sim \mathcal{N}(0, \sigma^2)$ with standard deviation $\sigma = 0.5$.

The updated SNR values are then clipped to remain within $[\text{SNR}_{\text{min}}, \text{SNR}_{\text{max}}]$.

3) Measurement Update: The weights are updated using Bayes' rule based on the binary feedback f (where $f = 1$ for NACK, $f = 0$ for ACK) received for the previously chosen MCS m. The likelihood L_i of the observation given particle i's SNR hypothesis is:

- If NACK ($f = 1$): $L_i = \text{BLER}(s_i(t), m)$
- If ACK ($f = 0$): $L_i = 1 - \text{BLER}(s_i(t), m)$

To prevent numerical instability, the likelihoods are clamped: $L_i = \max(L_i, \epsilon)$. The unnormalized weights are updated as:

$$w_i^{\text{unnorm}} = w_i(t-1) \cdot L_i$$

The weights are then normalized so they sum to 1. If the sum of the unnormalized weights is zero or invalid, they are reset to a uniform distribution $w_i(t) = 1/N_{\text{particles}}$. Otherwise:

$$w_i(t) = \frac{w_i^{\text{unnorm}}}{\sum_j w_j^{\text{unnorm}}}$$

4) Systematic Resampling: To prevent particle degeneracy, the effective sample size N_{eff} is calculated:

$$N_{\text{eff}} = \frac{1}{\sum_i w_i(t)^2}$$

If $N_{\text{eff}} < N_{\text{particles}}/2$, systematic resampling is triggered. Particles with high weights are duplicated, and particles with low weights are dropped. After resampling, all weights are reset to uniform: $w_i(t) = 1/N_{\text{particles}}$.

5) Candidate Feasibility Evaluation: To select the next MCS, the algorithm computes the expected BLER for every possible candidate MCS c across the current particle distribution:

$$\mathbb{E}[\text{BLER}_c] = \sum_i w_i(t) \cdot \text{BLER}(s_i(t), c)$$

A candidate MCS c is considered "feasible" only if its expected BLER is less than or equal to the target BLER T_{BLER}.

6) One-Step Look-ahead Reward Maximization: For each feasible candidate MCS c, the algorithm evaluates the expected total reward (throughput).

- Immediate Expected Reward:

$$R_{\text{immediate}}(c) = \sum_i w_i(t)\, c \left(1 - \text{BLER}(s_i(t), c)\right)$$

- Future Expected Reward (Look-ahead): The algorithm branches into two hypothetical futures (receiving an ACK or a NACK).

$$P_{\text{ACK}} = \sum_i w_i(t)\big(1 - \text{BLER}(s_i(t), c)\big),$$
$$P_{\text{NACK}} = 1 - P_{\text{ACK}}$$

It computes hypothetical future particle weights for both branches:

$$w_i^{\text{ACK}} = w_i(t) \cdot \frac{1 - \text{BLER}(s_i(t), c)}{P_{\text{ACK}}},$$
$$w_i^{\text{NACK}} = w_i(t) \cdot \frac{\text{BLER}(s_i(t), c)}{P_{\text{NACK}}}$$

Using these distributions, the algorithm determines a "safe" future MCS for the ACK branch ($m_{\text{safe}}^{\text{ACK}}$) and the NACK branch ($m_{\text{safe}}^{\text{NACK}}$) using the conservative fallback strategy (see below). The expected future rewards are:

$$R_{\text{ACK}} = \sum_i w_i^{\text{ACK}} m_{\text{safe}}^{\text{ACK}} \big(1 - \text{BLER}(s_i(t), m_{\text{safe}}^{\text{ACK}})\big),$$
$$R_{\text{NACK}} = \sum_i w_i^{\text{NACK}} m_{\text{safe}}^{\text{NACK}} \big(1 - \text{BLER}(s_i(t), m_{\text{safe}}^{\text{NACK}})\big)$$

- Total Expected Reward:

$$R_{\text{total}}(c) = R_{\text{immediate}}(c) + P_{\text{ACK}} R_{\text{ACK}} + P_{\text{NACK}} R_{\text{NACK}}$$

The algorithm selects and returns the candidate MCS c that maximizes $R_{\text{total}}(c)$.

7) Conservative Fallback: If no candidate MCS satisfies the target BLER in Step 5 (or when determining safe future steps in Step 6), the algorithm defaults to a conservative estimate. It finds the p_{cons}-th percentile (0.2) of the current weighted SNR distribution, subtracts the Δ_{margin} (0.5 dB) safety margin, and returns the highest MCS that satisfies the target BLER T_{BLER} for that specific pessimistic SNR value.

B. GPT 5.4

This algorithm implements a grid-based discrete Markov filter (histogram filter) paired with an asymmetric OLLA mechanism. It discretizes the SNR space into a fixed grid and tracks the probability distribution of the true SNR over time using a sliding window of up to 64 past transmissions. At each step, the distribution is propagated via Gaussian convolution (modeling a random walk) and updated through Bayesian inference based on ACK/NACK feedback. A trimmed mean of the posterior distribution yields a robust point estimate, discarding the extreme tails. Finally, the estimate is adjusted by an asymmetric OLLA offset—stepping up for ACKs and more aggressively down for NACKs—and a conservative warm-up margin is applied before selecting the MCS.

Let the target Block Error Rate be T_{BLER}. Let the history of observations be a sequence of (f_i, m_i) pairs, where f_i is the binary feedback (1 for NACK, 0 for ACK) and m_i is the transmitted MCS at step i. The algorithm processes an observation window of length $N_{\text{hist}} \leq 64$. If the history of observations is empty ($N_{\text{hist}} = 0$), the algorithm bypasses all tracking and directly returns the highest MCS index satisfying the target BLER for a hardcoded estimate of 7.0 dB.

1) Grid Setup and Prior Initialization: The continuous SNR space is discretized into a fixed grid G of $N_{\text{grid}} = 169$ points, linearly spaced from -12.0 dB to 30.0 dB. The spacing between points is $dx = 0.25$ dB. A pre-computed lookup table, $\text{BLER}_{\text{table}}(s, m)$, stores the BLER for every grid point $s \in G$ and MCS m.

The initial probability distribution $P_0(s)$ over the grid is set to a normalized discrete Gaussian prior centered at $\mu_{\text{prior}} = 9.0$ dB with a standard deviation of $\sigma_{\text{prior}} = 6.5$ dB. The active

tracking distribution is initialized as $P(s) \leftarrow P_0(s)$, defined as:

$$P_0(s) = \frac{\exp\left(-0.5\left(\frac{s-\mu_{\text{prior}}}{\sigma_{\text{prior}}}\right)^2\right)}{\sum_{x\in G}\exp\left(-0.5\left(\frac{x-\mu_{\text{prior}}}{\sigma_{\text{prior}}}\right)^2\right)}$$

2) Bayesian Grid Tracking: For each observation $i \in \{1, \ldots, N_{\text{hist}}\}$ in the history window, the tracking distribution $P(s)$ is sequentially updated in-place through four steps:

- Prediction (Convolution): The state undergoes a random walk modeled by convolving the distribution with a discrete, normalized Gaussian kernel K with standard deviation $\sigma_{\text{rw}} = 0.52$ dB:

$$K(\text{offset}) = \exp\left(-0.5\left(\frac{\text{offset}\cdot dx}{\sigma_{\text{rw}}}\right)^2\right)$$
$$P(s) \leftarrow (P * K)(s)$$

- Forget Mix (Regularization): To prevent assigning absolute zero probability to any state, a fraction $\alpha = 0.006$ of a uniform distribution is mixed in:

$$P(s) \leftarrow (1-\alpha)P(s) + \frac{\alpha}{N_{\text{grid}}}$$

- Measurement Update: The probabilities are scaled by the likelihood $L_i(s)$ of the observation f_i given the transmitted MCS m_i. The likelihood is clamped to $[10^{-7}, 1-10^{-7}]$:

$$L_i(s) = \begin{cases} \text{BLER}_{\text{table}}(s, m_i) & \text{if NACK } (f_i = 1) \\ 1 - \text{BLER}_{\text{table}}(s, m_i) & \text{if ACK } (f_i = 0) \end{cases}$$
$$P(s) \leftarrow P(s)\cdot L_i(s)$$

- Normalization: The distribution is normalized. If the sum $S = \sum_{s\in G} P(s)$ is zero or invalid, the filter resets:

$$P(s) \leftarrow \begin{cases} P(s)/S & \text{if } S > 0 \\ P_0(s) & \text{otherwise} \end{cases}$$

3) Posterior Trimmed Mean: After processing the history, the algorithm extracts a single representative SNR value from the final updated distribution $P(s)$. Instead of a standard mean, it computes a trimmed mean to ignore misleading probability tails:

- Compute the cumulative distribution function, CDF(s_k), of P over the grid points $s_k \in G$.
- Let k_{lo} and k_{hi} be the integer indices of the grid corresponding to the 15th percentile ($q_{\text{lo}} = 0.15$) and 85th percentile ($q_{\text{hi}} = 0.85$) of the CDF, respectively.
- The base SNR estimate is the weighted center of mass strictly between these two bounds:

$$S_{\text{base}} = \frac{\sum_{k=k_{\text{lo}}}^{k_{\text{hi}}} P(s_k)\cdot s_k}{\sum_{k=k_{\text{lo}}}^{k_{\text{hi}}} P(s_k)}$$

- If the trimmed weight sum is invalid or zero, it falls back to the median (50th percentile) of P.

4) Asymmetric OLLA Offset: An OLLA offset is accumulated over the entire available history, stepping the SNR estimate up slightly for every ACK and down more aggressively for every NACK.

- Up-step (for ACKs): $\text{step}_{\text{up}} = 0.0237$ dB
- Down-step (for NACKs): $\text{step}_{\text{down}} = \text{step}_{\text{up}} \cdot \frac{1-T_{\text{BLER}}}{\max(T_{\text{BLER}}, 10^{-9})}$

With total ACKs N_{ACK} and total NACKs N_{NACK}, the raw offset is calculated as:

$$\text{Offset}_{\text{raw}} = \text{step}_{\text{up}} \cdot N_{\text{ACK}} - \text{step}_{\text{down}} \cdot N_{\text{NACK}}$$

This offset is then clamped to $[-2.94,\ 0.25]$ dB, yielding $\text{Offset}_{\text{clamped}}$.

5) Warm-up Margin and Final Selection: When the observation history is short and the filter is not yet converged, a conservative warm-up margin is applied based on the total number of observations n:

- $n < 5$ observations: Margin $= -0.95$ dB
- $n < 10$ observations: Margin $= -0.35$ dB
- Otherwise: Margin $= 0.0$ dB

The final estimated SNR is the sum of the base trimmed mean, the OLLA offset, and the warm-up margin:

$$S_{\text{final}} = S_{\text{base}} + \text{Offset}_{\text{clamped}} + \text{Margin}$$

S_{final} is clamped to the grid boundaries $[-12.0,\ 30.0]$ dB.

The algorithm then uses the pre-computed BLER curves to return the highest MCS index whose expected BLER at S_{final} is $\leq T_{\text{BLER}}$.